

Deadlock: Giải thuật Banker

Kế hoạch phân công cho nhóm 8 thành viên: mỗi người một đầu việc đo đếm được, có sản phẩm bàn giao và tiêu chí nghiệm thu rõ ràng — bám sát đúng 6 mục mà cột “Nội dung” của đề bài yêu cầu.

PHẦN	ĐỀ TÀI	ĐỐI CHỨNG	SẢN PHẨM	
Deadlock	Giải thuật Banker	Đồ thị phân bổ tài nguyên	Quyển báo cáo + phần mềm GUI	
NHÂN LỰC				
8 thành viên · 4 tuần				

01

Giải mã đề bài

Cột “Nội dung” áp dụng chung cho cả 10 đề tài. Dưới đây là bản dịch sang đầu việc cụ thể cho đề tài Banker — đây chính là khung chấm điểm.

YÊU CẦU ĐỀ BÀI	PHẢI NỘP CÁI GÌ	PHỤ TRÁCH
A1. Lý thuyết	Deadlock là gì, 4 điều kiện Coffman, 4 hướng xử lý, khái niệm trạng thái an toàn — chuỗi an toàn	TV2
A2. Thuật toán	Cấu trúc dữ liệu, mã giả 2 thủ tục, ví dụ chạy tay từng bước, độ phức tạp	TV3
A3. So sánh với 1 giải thuật khác cùng Phần	Banker ↔ giải thuật đồ thị phân bổ tài nguyên: bảng tiêu chí + thực nghiệm đo	TV7
A4. Code giải thuật	Engine sạch, có chú thích, chạy được, kèm unit test	TV4
A5. Kết quả	Bộ test case, ảnh chụp, bảng đối chiếu kỳ vọng / thực tế, nhận xét	TV8
B. Phần mềm có giao diện	Ứng dụng GUI đóng gói chạy được, đầy đủ chức năng của thuật toán	TV5 · TV6 · TV1
Kiến trúc & vận hành	Hợp đồng dữ liệu, thiết kế kiến trúc, module xuất báo cáo, ghép quyền, slide, bảo vệ	TV1

Điểm mấu chốt hay bị bỏ sót: mục A3 yêu cầu so sánh với *một giải thuật khác của cùng Phần*. Phần Deadlock chỉ có 2 đề tài, nên đối tượng so sánh bắt buộc là **giải thuật đồ thị phân bổ tài nguyên** — không phải FCFS hay LRU. Nhóm nào so sai mục này mất trọn một chương.

02 Phân công 8 thành viên

Nguyên tắc chia: mỗi người sở hữu trọn vẹn một sản phẩm để không ai làm hộ ai, nhưng các đầu việc được xếp theo thứ tự phụ thuộc để tuần 1 không ai phải ngồi chờ.

TV1 Nhóm trưởng · Kiến trúc phần mềm

QUẢN TRỊ · CHUYÊN MÔN · GHÉP SẢN PHẨM · BẢO VỆ

PHẦN CHUYÊN MÔN

- **Viết hợp đồng dữ liệu** — file `banker_types` định nghĩa lớp `BankerState`, `StepLog`, `RequestResult`, enum `Verdict` và định dạng file `.json`. Xong **trước ngày thứ 3** để TV4, TV5, TV6 code song song mà ghép vào là chạy.
- **Viết Chương 4.1 — Kiến trúc chương trình:** sơ đồ khối ba lớp, biểu đồ ca sử dụng, biểu đồ lớp, biểu đồ tuần tự cho kịch bản “gửi yêu cầu tài nguyên”.
- **Cài module xuất báo cáo** trong phần mềm: nút “Xuất kết quả” ghi ra PDF/Excel gồm ma trận đầu vào, chuỗi an toàn và toàn bộ nhật ký từng bước.
- Viết Lời nói đầu, Kết luận và Hướng phát triển.

PHẦN QUẢN TRỊ

- Chốt công nghệ ngay ngày đầu; lập repo Git, quy ước nhánh và commit, viết `README` hướng dẫn chạy.
- Lập bảng Kanban, họp 2 buổi mỗi tuần, ghi **biên bản họp** và **bảng phân công** đưa vào phụ lục.
- Duyệt pull request — không ai đẩy thẳng vào nhánh chính.
- Ghép quyển: thống nhất định dạng, đánh số hình và bảng, mục lục tự động, chuẩn hoá trích dẫn, rà đạo văn.
- Đóng gói bản chạy kèm README và dữ liệu mẫu, **thử trên máy chưa cài môi trường lập trình**; làm slide và tổng duyệt.

BÀN GIAO

Hợp đồng dữ liệu · Chương 4.1 + 4 biểu đồ thiết kế · Module xuất báo cáo · Repo Git · Quyển báo cáo đã ghép · Bản đóng gói + slide + biên bản họp

TV2

Chương lý thuyết Deadlock

NỘI DUNG A1 · 8-10 TRANG

- Định nghĩa deadlock, phân biệt với starvation và livelock.
- **4 điều kiện Coffman**: loại trừ tương hỗ, giữ và chờ, không trưng dụng, chờ vòng tròn — mỗi điều kiện một ví dụ đời thực.
- Hai ví dụ kinh điển: hai tiến trình khoá chéo semaphore, và bài toán năm triết gia ăn cơm.
- Bốn hướng xử lý và vị trí của Banker: đây là **deadlock avoidance**, không phải prevention hay detection.
- Giải thích “thuật toán đà điều” và vì sao Windows, Linux thực tế chọn cách này.
- Trạng thái an toàn / không an toàn / deadlock và quan hệ bao hàm giữa chúng.
- Tự vẽ toàn bộ hình bằng draw.io, nộp kèm file nguồn — không chụp lại ảnh từ sách hay trên mạng.

BÀN GIAO

Chương 1 (.docx) + tối thiểu 5 hình tự vẽ + danh mục tài liệu tham khảo

TV3

Đặc tả thuật toán Banker

NỘI DUNG A2 · XƯƠNG SỐNG CỦA BÀI

- Bối cảnh: Dijkstra 1965, phép ví von với ngân hàng cho vay.
- Định nghĩa 4 cấu trúc dữ liệu **Available**, **Max**, **Allocation**, **Need**, các ràng buộc hợp lệ và quy ước so sánh vector.
- Viết mã giả **Safety Algorithm** và **Resource-Request Algorithm** (mục 03), kèm lưu đồ.
- Ví dụ chạy tay đầy đủ: bảng **Work / Finish** sau *từng* vòng lặp, không rút gọn.
- Ba kịch bản yêu cầu với ba lý do khác nhau: cấp phát, phải chờ, từ chối vì không an toàn.
- Chứng minh độ phức tạp $O(m \cdot n^2)$ và chỉ ra chuỗi an toàn không duy nhất.
- Bàn giao mã giả cho TV4 kèm 30 phút giải thích trực tiếp — không gửi file rồi thôi.

BÀN GIAO

Chương 2 (.docx) + file mã giả dùng chung + 2 lưu đồ thuật toán

TV4 Engine — mã nguồn lỗi

NỘI DUNG A4 · KHÔNG ĐỤNG ĐẾN GIAO DIỆN

- Cài đặt `BankerState` theo hợp đồng của TV1, với `is_safe()`, `request()`, `release()`, `finish()`.
- Hàm phải trả về **nhật ký từng bước** (chọn tiến trình nào, `Work` trước và sau, vì sao bỏ qua tiến trình khác) để giao diện chiếu lại — không chỉ trả về đúng/sai.
- Rollback nguyên tử khi trạng thái giả lập không an toàn; viết test chứng minh trạng thái không bị hỏng.
- Hàm kiểm tra hợp lệ dữ liệu, ném lỗi có mã và thông điệp tiếng Việt rõ nghĩa.
- Unit test tối thiểu 15 ca, chạy xanh 100%.
- Script đo thời gian chạy, xuất CSV cho TV7.
- **Điểm cộng:** liệt kê *tất cả* chuỗi an toàn bằng quay lui.
- Không import thư viện giao diện nào — engine phải chạy được từ dòng lệnh.

BÀN GIAO

Module engine độc lập + bộ unit test xanh + script đo hiệu năng

TV5 Giao diện — nhập liệu

NỘI DUNG B · NỬA TRÁI PHẦN MỀM

- Nhập số tiến trình `n`, số loại tài nguyên `m`, sinh bảng ma trận động.
- Lưới nhập `Available`, `Max`, `Allocation`; hỗ trợ dán dữ liệu từ Excel.
- Kiểm tra hợp lệ ngay khi gõ: ô sai tô đỏ kèm thông báo cụ thể, ví dụ "Ô P2-B: đã cấp 4 nhưng khai báo tối đa 3".
- Nút tính và khoá bảng `Need = Max - Allocation` (người dùng không được sửa tay).
- Dòng tổng `Total[j] = Available[j] + ΣAllocation[i][j]`.
- Mở / Lưu bộ dữ liệu `.json`, nạp nhanh 3 bộ mẫu, sinh dữ liệu ngẫu nhiên cho TV7 đo hiệu năng.

BÀN GIAO

Màn hình nhập liệu + 3 file dữ liệu mẫu + bộ sinh ngẫu nhiên

TV6

Giao diện — mô phỏng

NỘI DUNG B · PHẦN ĂN ĐIỂM DEMO

- Ô kết luận lớn: **AN TOÀN** kèm chuỗi **P1 → P3 → P4 → P2 → P0**, hoặc **KHÔNG AN TOÀN** kèm lý do.
- Bảng nhật ký: Bước · Work trước · Tiến trình chọn · Need · Allocation · Work sau.
- Hai chế độ **chạy từng bước** và **chạy tự động** có thanh chỉnh tốc độ, tô sáng tiến trình đang xét.
- Bảng **yêu cầu tài nguyên**: chọn tiến trình, nhập vector → ba kết cục với **ba câu lý do khác nhau**.
- Giải phóng tài nguyên khi tiến trình kết thúc; lịch sử trạng thái + nút Hoàn tác.
- Biểu đồ cột tài nguyên đã cấp / còn rảnh, cập nhật thời gian thực.
- **Điểm cộng**: vẽ đồ thị phân bổ tài nguyên — dùng lại được cho chương so sánh của TV7.

BÀN GIAO

Màn hình mô phỏng + đồ thị phân bổ + video demo 3–5 phút

TV7

Chương so sánh

NỘI DUNG A3 · MỤC DỄ MẤT ĐIỂM NHẤT

- Trình bày giải thuật đồ thị phân bổ tài nguyên: cạnh yêu cầu, cạnh cấp phát, **cạnh nhu cầu nét đứt**.
- Vẽ hai hình đối chứng: một trạng thái cấp phát được, một trạng thái tạo chu trình nên bị từ chối.
- Lập bảng so sánh 7 tiêu chí (mục 05) và giải thích *từng dòng*, không để bảng đứng trơ.
- Nêu rõ giới hạn cốt lõi: đồ thị chỉ đúng khi mỗi loại tài nguyên có **một thực thể duy nhất**; vẽ phản ví dụ chu trình mà không deadlock.
- Thực nghiệm: đo thời gian chạy khi **n** tăng dần, vẽ biểu đồ, đối chiếu với **$O(m \cdot n^2)$** . Số liệu phải đo thật.
- Kết luận nên dùng giải thuật nào trong hoàn cảnh nào.

BÀN GIAO

Chương 3 (.docx) + bảng số liệu đo + biểu đồ + 3 hình tự vẽ

Kiểm thử & chương kết quả

NỘI DUNG A5 · CHUẨN BỊ PHẢN BIỆN

- Bộ test case theo mục 04: mỗi ca ghi rõ đầu vào — kỳ vọng — thực tế — đạt/không, kỳ vọng lấy từ bảng chạy tay của TV3.
- Bổ sung tối thiểu 5 ca của riêng mình cho các tình huống lạ.
- Chụp màn hình chất lượng cao cho từng ca, đánh số hình thống nhất.
- Mở issue trên GitHub cho mỗi lỗi, theo dõi đến khi đóng — không báo miêng.
- Kiểm thử chéo: nhờ một người ngoài nhóm dùng thử 10 phút, ghi lại chỗ khó hiểu.
- Viết chương Kết quả: môi trường thử nghiệm, bảng kết quả, nhận xét, ưu điểm, hạn chế, hướng phát triển.
- Soạn bộ 15 **câu hỏi phản biện** kèm đáp án, tổ chức buổi hỏi chéo trước ngày bảo vệ.

BÀN GIAO

Chương 5 + bảng test case + thư mục ảnh + bộ Q&A phản biện

Đường găng: hợp đồng dữ liệu của TV1 và mã giả của TV3 chặn toàn bộ phần còn lại — hai việc này phải xong trong tuần 1, rồi TV4 làm engine, rồi TV5 và TV6 ghép giao diện, cuối cùng TV8 kiểm thử. TV2 và TV7 chạy song song, không nằm trên đường găng.

BẢNG LÀM VIỆC CHI TIẾT

Repo: github.com/BryannLee202/HDH_De_tai_9_Giai_thuat_Banker

TV	NHÁNH GIT	CODE?	KIẾN THỨC CẦN CÓ	THAO TÁC CỤ THỂ
TV1 Nhóm trưởng	tv1-kien-truc	Có Python vừa	Git nâng cao (nhánh, pull request, gộp mã) UML: sơ đồ khối, ca sử dụng, biểu đồ lớp, biểu đồ tuần tự Python: đọc ghi file, thư viện xuất PDF Word: mục lục tự động, đánh số hình theo chương	- git clone rồi git checkout tv1-kien-truc - Điền tên 8 người vào bảng trong README.md - Vẽ 4 biểu đồ bằng draw.io, lưu vào docs/ - Viết module xuất báo cáo trong gui/ - Commit, push, mở pull request, tự duyệt - Duyệt pull request của 7 người còn lại
TV2	tv2-chuong-ly-thuyet	Không	Lý thuyết deadlock: 4 điều kiện Coffman, 4 hướng xử lý Phân biệt deadlock / starvation / livelock Bài toán 5 triết gia, khoá chéo semaphore draw.io và cách trích dẫn tài liệu	Không cần cài gì. - Vào repo trên web, đổi nhánh sang tv2-chuong-ly-thuyet - Mở thư mục report/chuong-1-tv2/ - Add file → Upload files → kéo chuong-1.docx vào - Gõ mô tả rồi bấm Commit changes - Lập lại mỗi lần có bản mới
TV3 Ưu tiên số 1	tv3-dac-ta-thuat-toan	Không viết mã giả	Nắm chắc thuật toán Banker hơn cả người code Hai thủ tục: kiểm tra an toàn và xử lý yêu cầu So sánh vector theo từng thành phần Tính độ phức tạp $O(m \cdot n^2)$ Chạy tay ma trận không sai số	Không cần cài gì. - Vào web, đổi nhánh sang tv3-dac-ta-thuat-toan - Upload chuong-2.docx vào report/chuong-2-tv3/ - Phải có: mã giả 2 thủ tục, lưu đồ, bảng chạy tay đủ 5 vòng lặp, 3 kịch bản yêu cầu - Hợp 30 phút bàn giao mã giả cho TV4
TV4 Code nhiều nhất	tv4-engine	Có Python khá	Python: lớp, dataclass, mảng hai chiều, ngoại lệ Sao chép sâu — thiếu cái này là hỏng rollback Thư viện unittest Hiểu mã giả của TV3	- git clone rồi git checkout tv4-engine - Chạy python -m unittest discover -s tests -v → thấy 16 test đỏ - Mở engine/banker.py, thay từng TODO - Thứ tự: hợp lệ → an toàn → yêu cầu → giải phóng → tất cả chuỗi - Mỗi hàm xong là một commit + push - Xong khi 16 test xanh hết
TV5	tv5-gui-nhap-lieu	Có Python + PyQt5	PyQt5: bảng dữ liệu, ô nhập số, hộp thoại chọn file Bắt sự kiện khi người dùng gõ để kiểm tra ngay Đọc ghi JSON Sinh số ngẫu nhiên có ràng buộc	- git checkout tv5-gui-nhap-lieu Hợp với TV6 chốt bố cục hai vùng trước khi code - Tạo gui/cua_so_chinh.py, dựng khung cửa sổ - Làm lưới nhập, kiểm tra hợp lệ, tính Need - Làm menu Mở / Lưu, nạp 3 bộ mẫu trong data/ - Commit từng chức năng, không dồn một cục
TV6	tv6-gui-mo-phong	Có Python + PyQt5	PyQt5: bảng, bộ đếm thời gian, tín hiệu và khe cắm Tô màu và tô sáng ô trong bảng Vẽ hình: matplotlib hoặc vẽ tay trên khung Ngăn xếp lịch sử để làm nút Hoàn tác	- git checkout tv6-gui-mo-phong - Tuần 1 dựng bảng nhập ký với dữ liệu giả, chưa cần chờ TV4 - Ghép engine thật khi TV4 xong - Làm chạy từng bước, chạy tự động, panel yêu cầu - Làm nút Kết thúc tiến trình và Hoàn tác - Quay video demo, để trên Drive, không commit

TV	NHÁNH GIT	CODE?	KIẾN THỨC CẦN CÓ	THAO TÁC CỤ THỂ
TV7	tv7-chuong-so-sanh	Ít chạy script	Giải thuật đồ thị phân bổ tài nguyên và ba loại cạnh Phát hiện chu trình trong đồ thị có hướng Vì sao chu trình chưa chắc là deadlock Vẽ biểu đồ bằng Excel hoặc matplotlib	1. <code>git checkout tv7-chuong-so-sanh</code> (hoặc dùng web) 2. Vẽ 2 hình đồ thị đối chứng bằng draw.io 3. Lập bảng so sánh 7 tiêu chí, viết lời bình từng dòng 4. Chạy script đo của TV4 với $n = 10, 20, 50, 100, 200$ 5. Vẽ biểu đồ thời gian, đối chiếu với đường cong n^2 6. Upload <code>chuong-3.docx</code> vào thư mục của mình
TV8	tv8-kiem-thu	Có Python nhẹ	Cách viết một ca kiểm thử: đầu vào — kỳ vọng — thực tế Thư viện <code>unittest</code> ở mức đọc hiểu và thêm ca mới GitHub Issues để báo lỗi Chụp màn hình và trình bày bảng kết quả	1. <code>git checkout tv8-kiem-thu</code> 2. Đọc 16 ca có sẵn trong <code>tests/test_banker.py</code> 3. Thêm tối thiểu 5 ca của riêng mình 4. Chạy phần mềm thật, chụp màn hình từng ca 5. Mỗi lỗi mở một issue, theo đến khi đóng 6. Upload <code>chuong-5.docx</code> và soạn 15 câu phản biện

Ba người không cần cài đặt gì: TV2, TV3 và TV7 chỉ nộp file Word. Vào repo trên web, đổi sang nhánh của mình, mở thư mục chương của mình, bấm **Add file** → **Upload files**. Vẫn tính là commit mang tên họ, miễn là đăng nhập bằng tài khoản của chính mình.

03

Nội dung kỹ thuật lỗi

Nguồn đúng duy nhất cho TV3 và TV4, tránh tình trạng mã giả trong báo cáo một đăng, code một nẻo. Cả 8 người phải nắm được phần này.

CẤU TRÚC DỮ LIỆU

Với `n` tiến trình và `m` loại tài nguyên:

KÝ HIỆU	KÍCH THƯỚC	Ý NGHĨA
Available	m	Số thực thể mỗi loại còn rảnh trong hệ thống
Max	n × m	Nhu cầu tối đa mà tiến trình khai báo trước khi chạy
Allocation	n × m	Số thực thể đang được cấp cho tiến trình
Need	n × m	<code>Need = Max - Allocation</code> , phần còn có thể xin thêm

Ràng buộc phải kiểm tra khi nhập liệu: `0 ≤ Allocation[i][j] ≤ Max[i][j]` và `Total[j] = Available[j] + Σi Allocation[i][j]` là hằng số.

THỦ TỤC 1 — KIỂM TRA TRẠNG THÁI AN TOÀN

```
SafetyCheck(Available, Allocation, Need) → (an_toàn, chuỗi)
1. Work ← Available; Finish[i] ← false ∀i; chuỗi ← []
2. Tìm chỉ số i thoả: Finish[i] = false và Need[i] ≤ Work    (so sánh theo từng thành phần)
   Nếu không tìm được → sang bước 4
3. Work ← Work + Allocation[i]
   Finish[i] ← true; chuỗi.thêm(Pi); quay lại bước 2
4. Nếu Finish[i] = true ∀i → trả về (AN TOÀN, chuỗi)
   ngược lại                → trả về (KHÔNG AN TOÀN, ∅)

Độ phức tạp: O(m · n²)
```

THỦ TỤC 2 — XỬ LÝ YÊU CẦU TÀI NGUYÊN

RequestResources(i, Request) → CẤP PHÁT | CHỜ | LỖI

- Nếu Request > Need[i] → **LỖI**: xin vượt quá nhu cầu đã khai báo
- Nếu Request > Available → **CHỜ**: hệ thống không đủ tài nguyên rảnh
- Giả lập cấp phát:

Available ← Available - Request

Allocation[i] ← Allocation[i] + Request

Need[i] ← Need[i] - Request
- Chạy SafetyCheck trên trạng thái giả lập:

an toàn → giữ nguyên, **CẤP PHÁT**

không an toàn → **khôi phục** nguyên trạng bước 3, tiến trình phải **CHỜ**

Câu hay bị hỏi khi bảo vệ: trạng thái không an toàn *không* đồng nghĩa với deadlock. Nó chỉ có nghĩa hệ điều hành không còn *bảo đảm được* mọi tiến trình đều kết thúc trong tình huống xấu nhất — thực tế các tiến trình có thể không xin hết `Max` và vẫn chạy trót lọt. Banker chọn cách thận trọng: từ chối luôn.

VÍ DỤ CHUẨN DÙNG XUYÊN SUỐT

Năm tiến trình, ba loại tài nguyên với tổng số `A=10, B=5, C=7`. Cả nhóm dùng đúng bộ số này cho mã giả, cho test và cho ảnh demo để quyền báo cáo nhất quán.

P	ALLOCATION			MAX			NEED		
	A	B	C	A	B	C	A	B	C
P0	0	1	0	7	5	3	7	4	3
P1	2	0	0	3	2	2	1	2	2
P2	3	0	2	9	0	2	6	0	0
P3	2	1	1	2	2	2	0	1	1
P4	0	0	2	4	3	3	4	3	1

`Available = (3, 3, 2)` → trạng thái **an toàn**, một chuỗi an toàn là `<P1, P3, P4, P2, P0>`. Nhóm nên chỉ ra rằng chuỗi an toàn *không duy nhất* — ví dụ `<P1, P3, P4, P0, P2>` cũng hợp lệ. Đây là chi tiết ăn điểm.

04 Bộ test case bắt buộc

TV8 chạy đủ 10 ca dưới đây trên phần mềm và chụp màn hình từng ca. Bốn ca đầu chạy nối tiếp nhau trên cùng ví dụ chuẩn ở mục 03.

MÃ	TÌNH HUỐNG	KẾT QUẢ MONG ĐỢI
TC01	Trạng thái ban đầu, Available = (3,3,2)	An toàn — chuỗi <P1,P3,P4,P2,P0>
TC02	P1 yêu cầu (1,0,2)	Cấp phát — sau đó vẫn an toàn, Available = (2,3,0)
TC03	Tiếp TC02, P4 yêu cầu (3,3,0)	Chờ — vượt quá Available (3 > 2)
TC04	Tiếp TC02, P0 yêu cầu (0,2,0)	Từ chối — đủ tài nguyên nhưng dẫn tới trạng thái không an toàn
TC05	P2 yêu cầu (7,0,0) khi Need[2] = (6,0,0)	Lỗi — xin vượt quá Max đã khai báo
TC06	Available = (0,0,0) và mọi Need > 0	Không an toàn — không tiến trình nào chạy tiếp được
TC07	Mọi Need = 0	An toàn ngay, chuỗi là thứ tự bất kỳ
TC08	Biên: m = 1 và n = 1	Chạy đúng, không lỗi chỉ số mảng
TC09	Nhập Allocation > Max, nhập chữ, hoặc để trống ô	Chặn ngay tại giao diện, báo rõ ô nào sai
TC10	Sinh ngẫu nhiên n = 50, 100, 200	Không treo giao diện; ghi lại thời gian chạy cho TV7

05

Khung chương so sánh

Bộ tiêu chí TV7 dùng để dựng bảng. Mỗi dòng trong báo cáo cần một đoạn giải thích kèm theo, không để bảng đứng trơ.

Tiêu chí	Giải thuật Banker	Đồ thị phân bổ tài nguyên
Số thực thể mỗi loại tài nguyên	Nhiều thực thể	Chỉ áp dụng khi mỗi loại có đúng một thực thể
Cách biểu diễn	Bốn ma trận số	Đồ thị có hướng: đỉnh tiến trình, đỉnh tài nguyên, ba loại cạnh
Điều kiện an toàn	Tồn tại ít nhất một chuỗi an toàn	Biến cạnh nhu cầu thành cạnh cấp phát mà không tạo chu trình
Độ phức tạp mỗi lần kiểm tra	$O(m \cdot n^2)$	$O((n + m)^2)$ cho phát hiện chu trình
Tính trực quan	Khó hình dung, phải đọc bảng số	Rất trực quan, nhìn hình thấy ngay chu trình
Khả năng mở rộng	Tốt với hệ thống nhiều tài nguyên cùng loại	Đồ thị rồi rất nhanh khi số đỉnh tăng
Hạn chế chung	Cả hai đều đòi hỏi biết trước nhu cầu tối đa, số tiến trình cố định, và đều hy sinh hiệu suất do cấp phát quá thận trọng — lý do hệ điều hành thực tế hầu như không dùng deadlock avoidance	

Chi tiết ghi điểm: trong đồ thị nhiều thực thể, có chu trình *chưa chắc* đã deadlock — chu trình chỉ là điều kiện cần. Đây chính là lý do tồn tại giải thuật Banker. Nêu được điều này là điểm phân biệt nhóm hiểu bài với nhóm chép bài.

06 Phần mềm cần có gì

Đề bài ghi “đầy đủ các yêu cầu của thuật toán” — nghĩa là chỉ kiểm tra an toàn thôi là chưa đủ, phải mô phỏng được cả chu trình xin, cấp phát và giải phóng tài nguyên.

CÔNG NGHỆ ĐỀ XUẤT

Ưu tiên **Python 3 + PyQt5**: bảng ma trận dựng nhanh bằng widget bảng có sẵn, đóng gói một tệp thực thi bằng PyInstaller, vẽ biểu đồ bằng matplotlib. Nếu nhóm mạnh Java thì dùng **JavaFX**, mạnh .NET thì dùng **C# WinForms**. Điều quan trọng không phải chọn cái nào, mà là chốt trong ngày đầu và không đổi.

DANH SÁCH CHỨC NĂNG NGHIỆM THU

- ☐ Nhập được `n`, `m` tùy ý, sinh bảng ma trận động TV5
- ☐ Bảng `Need` tự tính, không cho sửa tay TV5
- ☐ Kiểm tra hợp lệ dữ liệu, tô đỏ và báo rõ ô sai TV5
- ☐ Lưu / mở bộ dữ liệu, dữ liệu mẫu, sinh ngẫu nhiên TV5
- ☐ Kiểm tra trạng thái an toàn, hiển thị chuỗi an toàn tìm được TV6
- ☐ Nhật ký từng bước: chọn tiến trình nào, `Work` trước và sau, vì sao bỏ qua tiến trình khác TV6
- ☐ Chế độ chạy từng bước và chạy tự động có chỉnh tốc độ TV6
- ☐ Mô phỏng yêu cầu tài nguyên với ba kết cục kèm ba lý do riêng TV6
- ☐ Giải phóng tài nguyên khi tiến trình kết thúc; hoàn tác trạng thái TV6
- ☐ *Điểm cộng*: liệt kê tất cả chuỗi an toàn TV4
- ☐ *Điểm cộng*: vẽ kèm đồ thị phân bố tài nguyên TV6
- ☐ *Điểm cộng*: xuất kết quả ra PDF hoặc Excel ngay từ phần mềm TV1

Tiến độ 4 tuần

Mốc thời gian giả định 4 tuần. Nếu thời hạn ngắn hơn thì nén tuần 1 và tuần 2 lại, tuyệt đối không cắt tuần 4.

TUẦN 1 – NỀN MÓNG

Chốt công nghệ và đặc tả

TV1 chốt công nghệ, dựng repo và viết xong **hợp đồng dữ liệu trước ngày thứ 3**. TV3 hoàn thành mã giả — chốt chặn của cả nhóm. TV2 viết xong chương lý thuyết. TV5, TV6 dựng khung giao diện với dữ liệu giả. **Cuối tuần:** họp duyệt mã giả trước cả nhóm.

TUẦN 2 – XÂY DỰNG

Engine chạy đúng, giao diện ghép được

TV4 hoàn thành engine và unit test xanh. TV5 xong màn hình nhập liệu. TV6 xong phần kiểm tra an toàn và bảng nhật ký. TV7 bắt đầu chương so sánh. TV1 viết Chương 4.1 và bốn biểu đồ thiết kế. **Cuối tuần:** engine ghép vào giao diện, chạy trọn ví dụ chuẩn.

TUẦN 3 – HOÀN THIỆN & KIỂM THỬ

Đóng bug và thu thập kết quả

TV6 xong mô phỏng yêu cầu, giải phóng và các chức năng điểm cộng. TV1 cài module xuất báo cáo, bắt đầu ghép quyển. TV8 chạy đủ test case, chụp ảnh, mở danh sách lỗi. TV7 chạy thực nghiệm đo hiệu năng. **Cuối tuần:** đóng băng tính năng, chỉ sửa lỗi.

TUẦN 4 – NỘP & BẢO VỆ

Đóng gói và tổng duyệt

TV1 hoàn thiện quyển, rà đạo văn, đóng gói bản chạy và thử trên máy sạch, làm slide. TV6 quay video demo. TV8 phát bộ câu hỏi phản biện, tổ chức buổi hỏi chéo. **Cuối tuần:** tổng duyệt trọn buổi bảo vệ, mỗi người trình bày đúng phần của mình.

08 Bẫy mất điểm

Những lỗi khiến một bài làm tốt vẫn bị trừ điểm khi chấm.

So sánh sai đối tượng. Dem Banker so với FCFS hay LRU là lạc đề — phải so với giải thuật đồ thị phân bổ tài nguyên, đề tài còn lại của Phần Deadlock.

Chỉ có nút “Kiểm tra an toàn”. Phần mềm không mô phỏng được chu trình yêu cầu — cấp phát — giải phóng thì chưa đạt yêu cầu “đầy đủ các yêu cầu của thuật toán”.

Kết quả chỉ hiện “an toàn / không an toàn”. Thiếu chuỗi an toàn và nhật ký từng bước thì giáo viên không có gì để đối chiếu với mã giả trong báo cáo.

Ba loại từ chối dùng chung một câu thông báo. Vượt quá **Need**, thiếu tài nguyên rảnh, và dẫn tới trạng thái không an toàn là ba lý do hoàn toàn khác nhau — gộp lại là dấu hiệu chưa hiểu thuật toán.

Chương trình không chạy trên máy người chấm. Phải nộp kèm bản đóng gói, README ghi rõ phiên bản và thư viện, cộng video demo dự phòng.

Hình chụp lại từ sách, văn bản chép nguyên khối. Hình phải tự vẽ và nộp kèm file nguồn; lý thuyết phải diễn đạt lại bằng lời của nhóm, có trích dẫn.

Chỉ một người trả lời được câu hỏi. Cả 8 thành viên phải nắm chắc mục 03, kể cả người chỉ làm giao diện. Giáo viên hỏi ngẫu nhiên chứ không hỏi theo phân công.

CÂU HỎI PHẢN BIỆN CẦN CHUẨN BỊ SẴN

- ☐ Vì sao trạng thái không an toàn chưa chắc đã là deadlock?
- ☐ Vì sao hệ điều hành thực tế hầu như không dùng giải thuật Banker?
- ☐ Nếu có tiến trình mới gia nhập hệ thống giữa chừng thì xử lý thế nào?
- ☐ Giải thuật Banker tác động lên điều kiện Coffman nào trong bốn điều kiện?

- ☐ Phân biệt deadlock prevention, avoidance và detection?
- ☐ `Need[i] = 0` có ý nghĩa gì với tiến trình đó?
- ☐ Vì sao độ phức tạp là $O(m \cdot n^2)$ chứ không phải $O(n^2)$?
- ☐ Chuỗi an toàn có duy nhất không? Chương trình chọn chuỗi nào và vì sao?
- ☐ Vì sao phải khôi phục trạng thái khi từ chối yêu cầu? Không khôi phục thì sao?
- ☐ Trong đồ thị nhiều thực thể, có chu trình thì chắc chắn deadlock không?

09

Cấu trúc quyền báo cáo

Bố cục và dung lượng gợi ý để TV1 ghép quyền, tổng khoảng 45–55 trang kể cả phụ lục.

PHẦN	NỘI DUNG	TRANG	PHỤ TRÁCH
Đầu quyền	Bìa, lời nói đầu, mục lục, danh mục hình và bảng, bảng phân công công việc	6	TV1
Chương 1	Tổng quan về deadlock	8–10	TV2
Chương 2	Giải thuật Banker: cấu trúc dữ liệu, mã giả, ví dụ chạy tay, độ phức tạp	10–12	TV3
Chương 3	So sánh với giải thuật đồ thị phân bổ tài nguyên	6–8	TV7
Chương 4	Cài đặt: kiến trúc chương trình (TV1), mã nguồn cốt lõi (TV4), thiết kế giao diện (TV5, TV6)	12–14	TV1, TV4, TV5, TV6
Chương 5	Kết quả thực nghiệm, bảng test case, ảnh minh hoạ, đánh giá	6–8	TV8
Cuối quyền	Kết luận, hướng phát triển, tài liệu tham khảo, phụ lục mã nguồn và biên bản họp	5	TV1

Tài liệu tham khảo nền tảng: Silberschatz, Galvin & Gagne — *Operating System Concepts*, chương Deadlocks. Trích dẫn đầy đủ trong quyền báo cáo theo một chuẩn thống nhất.